Latent Mamba: Adaptive Continuous Reasoning in Constant Memory

Kye Gomez
Swarms Corp
kye@swarms.world

November 9, 2026

Abstract

Chain-of-thought reasoning spends extra compute by writing tokens, and in a Transformer every token also grows the key–value cache. We describe *Latent Mamba*, an architecture that lets a Mamba-2 language model reason in continuous latent space for an input-dependent number of steps while its memory stays constant. The model feeds its normalised final hidden state back as its next input through a learned *thought projector* that adds a thinking-mode embedding, and a *halting head* decides when to stop. We observe that in a selective state-space model, depth recurrence with a shared state is the same computation as interleaving self-generated latent inputs along the sequence, which motivates placing thoughts in time rather than in depth. Training follows a Coconut-style curriculum and then a PonderNet objective; because the recurrent state has a fixed size, the answer can be scored after every thought step in a single batched pass. Fine-tuning a pretrained 130M-parameter Mamba-2 on ProsQA with this recipe, the model answers **99.0%** of held-out test questions while reasoning entirely in latent space with six thoughts, against 70.8% for the same model trained to write its chain of thought and 74.6% for the same model trained to answer directly. Replacing the thoughts with zeros or noise at test time drops accuracy to 77% or below, so the gain comes from what the thoughts contain. The learned halting head stops after 3.1 thoughts on average at 98.6% accuracy, matching a fixed budget of three, but adapts only weakly to problem difficulty.

1 Introduction

Language models solve multi-step problems better when they can compute before answering, usually by emitting a chain of thought (CoT) or a scratchpad [37, 27, 21]. This has two costs. Natural-language tokens are a narrow channel for intermediate state, and many of them serve fluency rather than reasoning [19]. In a Transformer [34], every generated token also adds an entry to the key–value (KV) cache, so memory grows with the length of the reasoning.

Latent reasoning moves the intermediate computation into the model's continuous hidden space [39]. Coconut feeds the last hidden state back as the next input embedding instead of decoding it [19]. Recurrent-depth models iterate a shared block a variable number of times per token [13]. Both let a model think without writing text, but in a Transformer each continuous thought still occupies a cache slot.

Selective state-space models (SSMs) [17] such as Mamba and Mamba-2 [18, 9] carry context in a fixed-size recurrent state instead of a cache, so a thought step costs constant memory: it updates the state and nothing else. We develop this into a concrete architecture, a training procedure, an implementation that trains from pretrained checkpoints, and a first evaluation.

Contributions.

- A *time–depth collapse* for SSMs (Proposition 1): depth recurrence with a shared state equals self-generated inputs interleaved along the sequence. Placing thoughts in time avoids the missing-state problem that per-iteration states create under adaptive exit.
- **Latent Mamba** (Figure 2): a thought projector with a thinking-mode embedding, a halting head, and functional fixed-size state handling. The added parameters are about 0.3% of a 2.7B model and 0.6% of a 130M model.

- A training recipe (Algorithms 1 and 2) and an adaptive inference procedure (Algorithm 3). PonderNet scoring is batched across thought steps, which is cheap only because SSM state is fixed-size.
- Exact batching of prompts of different lengths (Proposition 2) and a training path that cuts saved activations at batch 32 from 75 GiB to 4.2 GiB with identical gradients (Section 4.4).
- A reference implementation that loads pretrained Mamba-2 checkpoints without renaming, numerical verification of its kernels (Table 2), and an experiment on ProsQA (Section 6) where the fully latent model reaches 99.0% test accuracy against 70.8% (written CoT) and 74.6% (no CoT), with thought ablations, a thinking-budget sweep, learned halting, and measured memory and latency: with fused kernels and CUDA graphs, a thought costs 2.0 ms, against 6.8 ms for a GPT-2 decoding step.

2 Background

Mamba-2. Each layer projects its input u_t to a gate z_t, a branch (x_t, b_t, c_t) that passes through a causal depthwise convolution of width K, and per-head step sizes $\Delta_t = \mathrm{softplus}(w^\top u_t + \beta) > 0$. For head h with decay rate $a_h < 0$, head input $x_t \in \mathbb{R}^P$ and $b_t, c_t \in \mathbb{R}^N$,

$$S_t = e^{\Delta_t a_h} S_{t-1} + \Delta_t \, x_t b_t^\top, \qquad y_t = S_t c_t + D_h x_t, \tag{1}$$

followed by a gated RMSNorm [38] and an output projection [9]. The recurrent state of a layer is the convolution window (the last $K-1$ inputs) together with $S \in \mathbb{R}^{H \times P \times N}$. Training processes whole sequences with the chunked SSD algorithm, a blocked form of a parallel scan [6, 24]; inference applies Equation (1) one token at a time. Both compute the same function.

Continuous thought. Coconut [19] marks a latent span with `<bot>` and `<eot>`. Inside the span the final hidden state is fed back as the next input embedding. It is trained with a curriculum adapted from implicit CoT [11]: at stage k, the first k language reasoning steps are replaced by ck continuous thoughts, and the loss covers only the remaining text.

Adaptive computation. ACT [15] and PonderNet [4] let a network choose how many steps to run. PonderNet predicts, at each step n, a conditional halting probability λ_n. This gives the distribution $p_n = \lambda_n \prod_{j<n}(1 - \lambda_j)$. Training minimises $\sum_n p_n \mathcal{L}_n + \beta \, \mathrm{KL}(p \,\|\, \mathrm{Geo}(\lambda_p))$, where $\mathcal{L}_n$ is the task loss after step n.

3 Latent Mamba

3.1 Where should a recurrent model think?

A model can add computation along two axes [39, 2]: *depth* (iterate a block several times at one position) and *time* (insert extra positions). Recurrent-depth Transformers take the depth route [13]. In an SSM the depth route raises a question that attention does not: which recurrent state does iteration i at position t read?

If each iteration has its own state stream, adaptive exit breaks the streams (Figure 1a). When token t stops after one iteration, iterations two and up at token t+1 have no state from position t to read. If instead all iterations share one state, the problem disappears and the computation becomes recurrence in time.

Proposition 1 (Time–depth collapse)**.** *Let $F : (u, S) \mapsto (h, S')$ be one pass of an SSM stack over a single input, and let g map outputs to inputs. Suppose the model runs r_t iterations at position t. Iteration i+1 takes input $g(h_t^{(i)})$ and the state $S_t^{(i)}$ left by iteration i; iteration 1 at position t+1 takes u_{t+1} and $S_t^{(r_t)}$. Then every output and state equals that of a single pass of F over the interleaved sequence $\big(u_1, g(h_1^{(1)}), \ldots, g(h_1^{(r_1-1)}), u_2, \ldots\big)$.*

Proof sketch. Both procedures apply F to the same (input, state) pairs in the same order. Induct on the position in the interleaved sequence. □

The statement is elementary, but it fixes the design. In an SSM, shared-state depth recurrence *is* thinking in time. That form keeps memory constant, needs no new kernels, and has no missing-state problem, so Latent Mamba places its thoughts along the sequence (Figure 1b). The feedback map g is the thought projector.

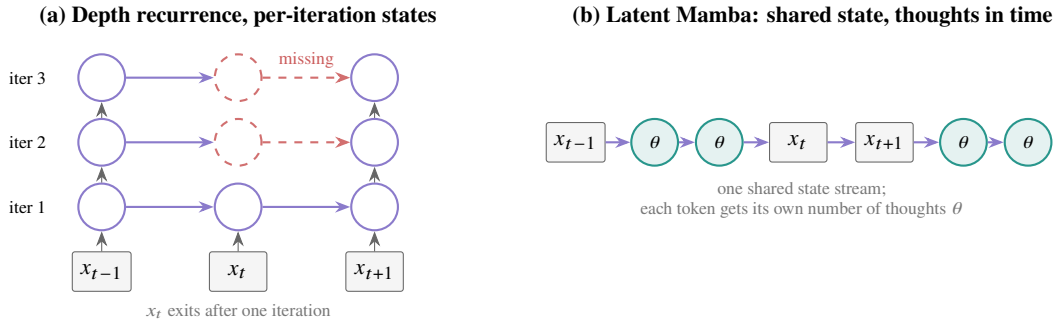


Figure 1: (a) Per-iteration state streams break under adaptive exit: when x_t halts after one iteration, the deeper iterations at x_{t+1} have no state to read. (b) With a shared state, depth recurrence becomes thoughts interleaved in time (Proposition 1). Purple arrows carry SSM state.

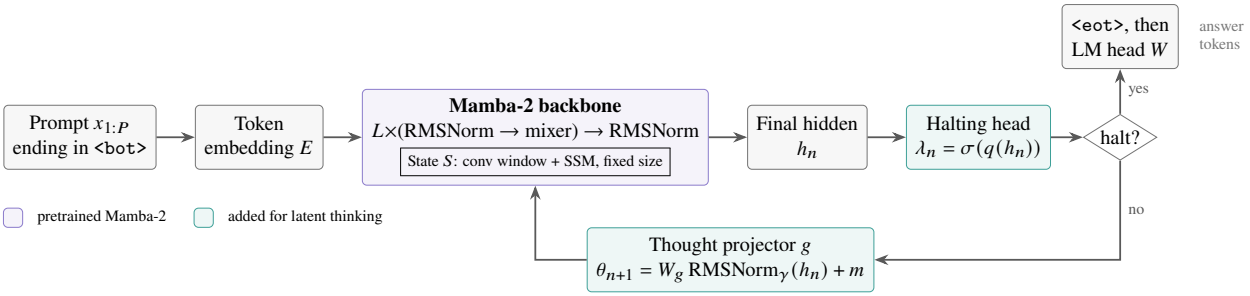


Figure 2: Latent Mamba. The prompt is prefilled with the parallel scan. Each thought feeds the projected final hidden state back as the next input and updates only the fixed-size state S. The halting head decides between another thought and decoding the answer after <eot>.

3.2 Architecture

The prompt ends in <bot>. Let h_0 be the final normalised hidden state at <bot>, S_0 the state after the prompt, and F one recurrent step of the whole backbone. Thought step $n \geq 1$ computes

$$\theta_n = W_g\,\mathrm{RMSNorm}_\gamma(h_{n-1}) + m, \tag{2}$$

$$(h_n, S_n) = F(\theta_n, S_{n-1}), \qquad \lambda_n = \sigma\big(q(h_n)\big). \tag{3}$$

After halting, the model reads <eot> and decodes the answer with the language-model head W. Table 1 lists the added parameters.

Thought projector. The final hidden state lives in output space, not input-embedding space. Mamba computes Δ_t, b_t and c_t from its input, so off-distribution inputs push these projections into regimes never seen in training. We therefore initialise $W_g = I$, $m = 0$ and γ to the RMS of the embedding matrix. At initialisation, the model is exactly Coconut with thoughts at the scale of a token embedding.

Thinking-mode embedding and prompt retention. By Equation (1), after n thoughts the part of head h's state that came from the prompt has been multiplied by

$$\prod_{k=1}^{n} e^{\Delta_{h,k} a_h} \;=\; \exp\big(a_h \textstyle\sum_{k=1}^{n} \Delta_{h,k}\big). \tag{4}$$

Long thinking can thus erase the question. $\Delta_{h,k}$ is a function of the input, so the learned vector m gives the model a direct way to select small step sizes while thinking. This retains the prompt, while thought content still enters through the convolution and the x and b paths. The cumulative step size $\sum_k \Delta_{h,k}$ is a cheap diagnostic of prompt decay.

Halting head. q is a two-layer MLP. Its last layer has zero weights and bias $\mathrm{logit}(\lambda_p)$, so the initial halting distribution equals the geometric prior and the initial KL term is zero.

Module	Params	2.7B (d=2560) Count	2.7B (d=2560) Share	130M (d=768) Count	130M (d=768) Share
Thought projector	$d^2 + 2d$	6.56M	0.24%	0.59M	0.46%
Halting head	$\frac{d^2}{4} + \frac{d}{2} + 1$	1.64M	0.06%	0.15M	0.11%
Total		8.20M	0.30%	0.74M	0.57%

Table 1: Parameters added to Mamba-2 models.

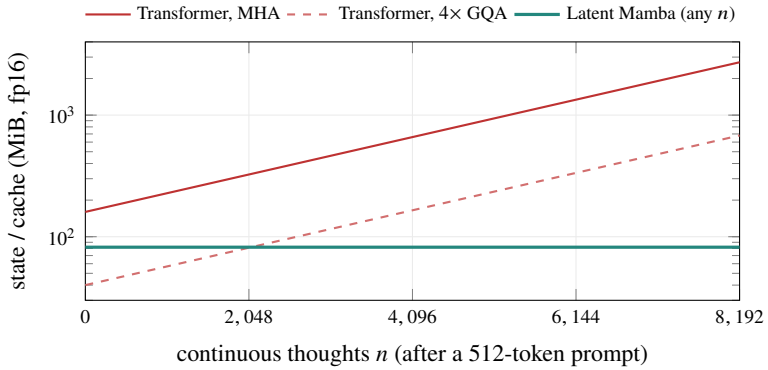


Figure 3: Analytic memory for a 2.7B-scale configuration at batch size 1. Transformer KV cache grows with every thought; the Latent Mamba state does not.

Why thoughts add expressivity. Within one pass, each layer's state update in Equation (1) is linear in S. A thought composes the full nonlinear network, including gates, normalisation and all layers, before the next state update. So n thoughts form a nonlinear recurrence of depth $n \cdot L$ in latent space, applied to whatever the state already holds.

3.3 Memory

The state of the whole model occupies

$$L\left[(K-1)(d_{\text{inner}} + 2N) + HPN\right] \tag{5}$$

values, independent of both prompt length and thought count. For the Mamba-2 2.7B configuration (L=64, d=2560, N=128, P=64, K=4) this is 671,488 values per layer, or 82.0 MiB in fp16. A 32-layer Transformer of the same width with multi-head attention stores 0.3125 MiB of KV cache per token; with 4× grouped-query attention [1], 0.078 MiB. Multi-query attention [32] shrinks the per-token cost further, but the cache still grows with every token.

Figure 3 compares the three after a 512-token prompt. The grouped-query Transformer is smaller until about 1,050 cached tokens, so the advantage is not universal. Beyond that point every Transformer thought costs memory, while Latent Mamba's footprint stays flat at any thinking budget. At the scale used in our experiments the comparison is starker: Mamba-2 130M (L=24, d_{inner}=1536, H=24) holds 9.25 MiB of state in bf16, while GPT-2 124M [29] (117M in the original count) stores 36 KiB of KV cache per token, so its cache already exceeds Latent Mamba's whole state after 263 tokens. Section 6.3 measures both on GPU.

3.4 Exact batching of prompts with different lengths

Real prompts differ in length. Padding them on the left would normally push pad tokens through the recurrent state. We avoid this with a mask that makes pad positions an exact identity on the state.

Proposition 2 (Exact left padding)**.** *Run a Mamba-2 layer from the zero state over a left-padded sequence. At every padded position, set the convolution input (x_t, b_t, c_t) to zero before the convolution, and set $\Delta_t = 0$ after the softplus.*

Algorithm 1 Curriculum loss with per-example thought counts

Require: prompts $x_{1:P}$ ending in <bot> (left padded, with mask); targets $y_{1:T}$; counts n_b

$(H, S) \leftarrow \textsc{Backbone}(E(x), \mathbf{0})$ ▷ chunked scan, masked

$h \leftarrow H_P$ ▷ hidden state at <bot>

for $i = 1, \ldots, \max_b n_b$ **do**

  $(h', S') \leftarrow F\big(W_g\,\mathrm{RMSNorm}(h) + m,\ S\big)$ ▷ one step

  $a_b \leftarrow [\, i \le n_b \,]$ for every example b

  $h \leftarrow a \odot h' + (1 - a) \odot h; \quad S \leftarrow a \odot S' + (1 - a) \odot S$

end for

$Z \leftarrow \textsc{Backbone}\big(E([\texttt{<eot>}, y_{1:T-1}]), S\big)$ ▷ teacher forcing

return $\mathrm{CE}(WZ, y)$ over non-padding targets

Algorithm 2 PonderNet step with batched branch scoring

Require: prompts x ending in <bot>; targets y; budget N; prior λ_p; weight β

$(H, S) \leftarrow \textsc{Backbone}(E(x), \mathbf{0}); \quad h \leftarrow H_P$

for $n = 1, \ldots, N$ **do**

  $(h, S) \leftarrow F\big(W_g\,\mathrm{RMSNorm}(h) + m,\ S\big)$

  $\lambda_n \leftarrow \sigma(q(h)); \quad S^{(n)} \leftarrow S$

end for

$\lambda_N \leftarrow 1; \quad p_n \leftarrow \lambda_n \prod_{j<n}(1 - \lambda_j)$ ▷ log space

$\bar{S} \leftarrow \textsc{Concat}_{\text{batch}}(S^{(1)}, \ldots, S^{(N)})$

$\ell_{1:N} \leftarrow$ per-example CE of y decoded from $\bar{S}$ ▷ one pass

$g_n \propto \lambda_p(1 - \lambda_p)^{n-1}$, normalised on $\{1..N\}$

return $\sum_n p_n \ell_n + \beta \sum_n p_n \log(p_n / g_n)$

Then the hidden states at real positions, and the final conv window and SSM state, equal those of the unpadded sequence. The same holds for the whole stack, applying the mask in every layer.

Proof sketch. With $\Delta_t = 0$, Equation (1) gives $S_t = S_{t-1}$, so the SSM state stays zero through the padding. The convolution window after the padding contains only zeros, which equals the zero initial window. Real positions therefore see exactly the state and window they would see without padding. Outputs at padded positions are arbitrary but never read: every later layer masks them again. □

Proposition 2 lets a batch mix prompts of any lengths with no bucketing. The implementation verifies it in float64 (Table 2).

4 Training and inference

4.1 Curriculum

We follow Coconut [19] (Algorithm 1). Stage 0 is language CoT after <bot><eot>, i.e. zero thoughts. At stage $k \le K$, the first k of an example's s reasoning steps are removed from the target and replaced by ck thoughts; as in Coconut, the thought count is ck even when $s < k$. A final stage $K{+}1$ removes every step and uses cK thoughts. Thought counts can differ within a batch. An example that has finished thinking is frozen by selecting its old state and hidden vector, so one batch runs $\max_b n_b$ steps. Gradients flow through every thought. For long budgets, the state can be detached after a fixed number of steps, as in recurrent-depth training [13]. We reset the optimiser at each stage, as Coconut does, and keep the checkpoint with the best validation accuracy within each stage.

Algorithm 3 Adaptive latent generation

Require: prompt x ending in `<bot>`; maximum N; threshold τ

$(H, S) \leftarrow \textsc{Backbone}(E(x), \mathbf{0})$; $\quad h \leftarrow H_P$

$\rho \leftarrow 1$; $\quad n \leftarrow 0$ ▷ ρ: probability of not halting

while $n < N$ **do**

  $(h, S) \leftarrow F\big(W_g\,\mathrm{RMSNorm}(h) + m,\ S\big)$; $\quad n \leftarrow n + 1$

  $\rho \leftarrow \rho\,(1 - \sigma(q(h)))$

  if $1 - \rho \geq \tau$ **then break**

  end if

end while

$v \leftarrow$ `<eot>`

repeat

  $(h, S) \leftarrow F(E(v), S)$; $\quad v \leftarrow \arg\max Wh$; $\quad$ emit v

until end token or length limit

4.2 Learning to halt

The final stage trains the halting head with the PonderNet objective (Algorithm 2). The model takes N thoughts and keeps the state after each. Because these states have a fixed size, the N branches can be concatenated along the batch axis, and the answer is scored from all of them in one pass of $N \cdot B$ short sequences. The extra cost is about N answer passes. In a Transformer, the same branching would copy N KV caches of growing length.

We set $\lambda_N = 1$ so that p is a proper distribution, and use the geometric prior truncated to $\{1, \ldots, N\}$. Two cheaper alternatives need no halting training: a zero-shot convergence rule, as used for recurrent-depth models [13, 33], and reinforcement learning with a per-thought compute penalty.

4.3 Adaptive inference

Algorithm 3 thinks until a halting rule fires and then decodes. The learned rule stops at the median of the predicted halting distribution, when $1 - \prod_j (1 - \lambda_j) \geq \tau$ with $\tau = 0.5$. The convergence rule stops when $\|h_n - h_{n-1}\| / \|h_{n-1}\| < \epsilon$. A fixed rule runs exactly N thoughts, matching the curriculum stages. Within a batch, halted examples are frozen while the others continue, and memory is constant throughout.

4.4 Training at scale

Our first full-size run ran out of memory on a 93 GiB GPU at batch 32. We measured the activations saved for the backward pass on the longest ProsQA prompts (592 tokens) and made two changes.

No per-head copies of b and c. Mamba-2 checkpoints use a single group, so b_t and c_t are shared by all heads. A direct transcription of the chunked SSD algorithm broadcasts them to every head before the contractions, which materialises H copies (24 for the 130M model). We instead contract $c_l^\top b_s$ once per chunk and apply the per-head decay afterwards, and fold the per-head decay into x before forming chunk states. This removes roughly 30% of the saved bytes per token and leaves results unchanged (Table 2).

Activation checkpointing. We recompute each Mamba layer during the backward pass instead of storing its internals [7]. Because state updates are functional, a layer is a pure function of (input, state, mask) and checkpointing needs no special handling. Saved activations for a batch of 32 of the longest prompts fall from 75.0 GiB to 4.2 GiB. Loss and gradients are identical (maximum relative difference 0 in float32), at the cost of about one extra forward pass.

Precision. We train under bf16 autocast [26] but run the selective scan and the single-step recurrence in float32: the scan exponentiates cumulative sums of Δa, which bf16 represents poorly. Section 5 describes optional fused kernels, each checked against this reference before use.

Check (float64)	Max. abs. error
Chunked scan vs. recurrence, outputs (batch 3, length 21, H=4, P=8, N=16, chunk 8)	1.1×10^{-14}
Chunked scan vs. recurrence, final state	2.7×10^{-15}
Prefill + per-token continuation vs. one pass, logits	2.2×10^{-16}
final state after continuation	1.4×10^{-17}
Left-padded vs. unpadded prompt, hidden state (Proposition 2)	2.2×10^{-16}
conv window and SSM state	2.2×10^{-16}
Halting distribution sums to one	0
Activation checkpointing vs. none, gradients (float32, relative)	0

Table 2: Numerical checks of the implementation. All but the last are part of the file's built-in self-test, which also checks that gradients reach the thought projector and the halting head, that a stopped example stays frozen while others continue, and that generation runs under every halting rule.

5 Reference implementation

The model is written in plain PyTorch. Sequences use the chunked SSD algorithm [9] started from an arbitrary initial state; single tokens use the exact recurrence of Equation (1). State updates are functional: they return new tensors, so a cache can be branched, frozen per example or concatenated without side effects, as Algorithms 1 and 2 require. The file also contains a diagnostic ablation that replaces thought content with zeros, which turns thoughts into pause tokens [14], or with noise of matching RMS.

Fast paths. On CUDA the training script replaces parts of the reference by default. Prompts and answers use the fused chunked scan of `mamba_ssm`, forward and backward. In inference, thoughts and decoded tokens use the fused single-step kernels of `mamba_ssm` and `causal-conv1d`, and each step is captured once per batch shape as a CUDA graph and replayed, which removes most kernel-launch overhead. The fused single-step kernels have no backward pass, so training thoughts instead use the reference step compiled with `torch.compile`. Each fast path is checked against the reference at startup (forward, and backward where used in training) and falls back to it on disagreement. The accuracy results below use the reference implementation; the latency results of Section 6.3 use the fast paths.

Pretrained weights. The mixer follows Mamba-2's parameterisation (input projection to $[z, x, b, c, \Delta]$, depthwise causal convolution over (x, b, c), per-head A, Δ bias and D, gated RMSNorm), and the parameter names of the public `state-spaces/mamba2-*` checkpoints, pretrained on 300B tokens of the Pile [12, 9], already match ours, so they load without renaming; only the two new modules are initialised fresh. As a sanity check, the ported 130M model scores 2.63 nats per token on a held-out Wikipedia paragraph and continues "The capital of France is" fluently. The three special tokens (`<bot>`, `<eot>`, pad) use spare rows of the checkpoint's padded vocabulary and are initialised from the embedding of "<<", as Coconut does.

Verification. The method depends on one property: continuing from a stored state must give exactly the result of an uninterrupted pass. Table 2 reports the checks, run in float64 on the current implementation. The scan uses three chunks with end padding and a random initial state.

6 Experiments

6.1 Setup

Task. ProsQA [19] asks whether an entity belongs to one of two fictional concepts, given a randomly generated graph of "every A is a B" facts. Answering requires finding a path through the graph, with distracting branches, so it rewards planning and search. Each question has two candidate answers, so chance is 50%. We use Coconut's splits (17,886 / 300 / 500 examples). With the GPT-NeoX tokenizer [5], questions are long (median 357 tokens, 99th percentile 502, maximum 591); the written reasoning plus answer is short (median 50 tokens). Questions need 3 to 6 gold reasoning steps (124, 133, 36 and 7 validation questions and 202, 217, 69 and 12 test questions respectively).

Base model	`state-spaces/mamba2-130m`	Optimiser	AdamW [23], lr 10^{-4} (constant), wd 0.01
Hardware	1× H100 NVL (94 GB)	Batch	32, gradient clipping 1.0
Precision	bf16 autocast, float32 scan	Optimiser reset	every stage
Chunk length	64	Checkpoint	best validation accuracy per stage
Thoughts per step c	1	Halting	$N = 6$, $\lambda_p = 0.2$, $\beta = 0.01$
Latent stages K	6	Seed	0 (one run per model)

Table 3: Training settings for the ProsQA experiments.

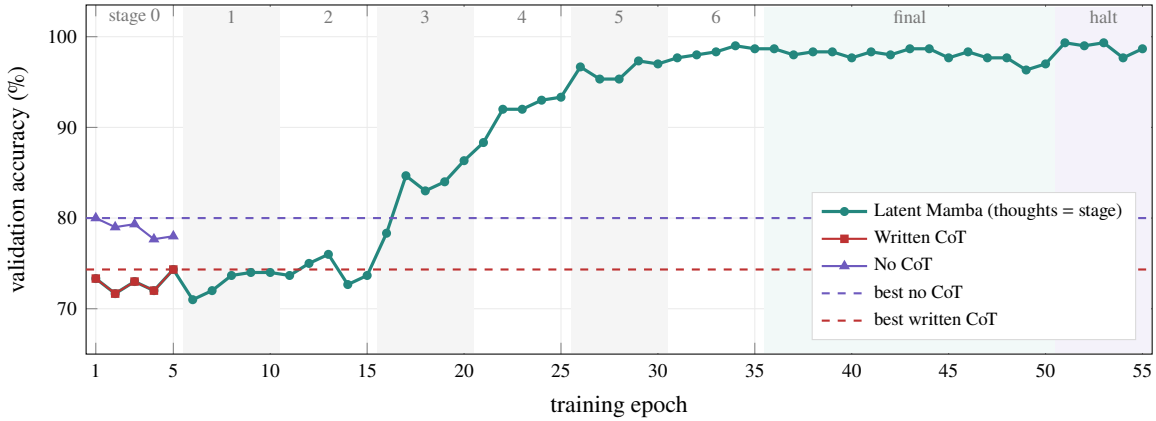


Figure 4: Validation accuracy on ProsQA (300 questions) after every epoch. Shaded bands mark curriculum stages; at stage k the model uses k thoughts in place of the first k reasoning steps, and the final stage replaces all steps with 6 thoughts. In the halting stage ("halt") the model chooses its own budget and uses about 3 thoughts. Stage 0 of the latent run is identical to the written-CoT run (same seed and data order).

Models. All three models start from the same pretrained Mamba-2 130M checkpoint and differ only in what they are trained to output:

- **No CoT:** question `<bot><eot> ### answer`.
- **Written CoT:** question `<bot><eot>` $\text{step}_1 \ldots \text{step}_s$ `### answer`.
- **Latent Mamba:** the full curriculum (stage 0 = written CoT, stages 1–6, final stage with all steps replaced by thoughts), then the halting stage.

Training. We follow Coconut's published ProsQA configuration: $c = 1$ thought per step, $K = 6$ latent stages, 5 epochs per stage including stage 0, and 15 epochs in the final stage, for 50 epochs in total; the halting stage adds 5 epochs. The baselines train for 5 epochs, matching stage 0. Table 3 lists the remaining settings. One difference from Coconut is the batch: we train on one GPU with batch 32, where Coconut used four GPUs with 32 each. Each phase continues from the weights at the end of the previous one, and the checkpoint kept for a phase is its best validation epoch, as in Coconut. Answers are decoded greedily (up to 128 tokens) and scored by exact match of the text after the last "`#`", Coconut's rule.

Evaluation. We report test accuracy on the 500 held-out questions for the kept checkpoint of each model; for Latent Mamba this is the halting-stage checkpoint (epoch 51). We run it in six modes: *learned* halting ($\tau = 0.5$, at most $N = 6$ thoughts); *fixed*, exactly 6 thoughts; *convergence* halting (relative change below 10^{-2}, at most 6); *no thoughts*, decoding directly after `<bot><eot>`; and two ablations that run 6 thoughts but replace the content of every thought with zeros or with noise of matching RMS. A sweep fixes the budget at every value from 0 to 12 thoughts.

6.2 Results

Figure 4 shows validation accuracy through training and Table 4 the best epoch of each stage.

Model / stage	Thoughts	Written reasoning steps	Best val. accuracy
No CoT	0	none	80.0
Written CoT	0	all	74.3
Latent Mamba, stage 1	1	all but the first	74.0
stage 2	2	all but the first two	76.0
stage 3	3	all but the first three	86.3
stage 4	4	…	93.3
stage 5	5	…	97.3
stage 6	6	none for $s \leq 6$	99.0
final (fully latent)	**6**	**none**	**98.7**
halting stage	learned, mean 3.1	none	**99.3**

Table 4: Best validation accuracy (%) per model and curriculum stage on ProsQA. One standard error at these sample sizes is about 2.5 points near 75% and under 1 point near 98%.

Model and inference mode	Thoughts (mean)	Test accuracy
No CoT	0	74.6
Written CoT	0	70.8
Latent Mamba, learned halting	3.11	98.6
fixed budget	6	**99.0**
convergence halting	6.00	**99.0**
Ablations, same checkpoint		
no thoughts	0	73.6
6 thoughts, content set to zero	6	76.8
6 thoughts, content replaced by noise	6	75.8

Table 5: Test accuracy (%) on ProsQA (500 questions). All Latent Mamba rows use the same halting-stage checkpoint. One standard error is about 2 points near 74% and 0.5 points near 99%.

Latent reasoning beats both baselines by a wide margin on held-out data. On the test set the latent model answers 99.0% of questions with six thoughts, 28.2 points above written CoT and 24.4 points above answering directly (Table 5). For Latent Mamba, validation and test agree to within a point. The baselines lose 3.5 (written CoT) and 5.4 (no CoT) points from validation to test, as expected when the best of five noisy validation epochs is selected, so the gap widens on held-out data. Coconut reports 97.0% on the ProsQA test set with GPT-2 [19], so a 130M Mamba-2 with a constant-size state reaches the same range. That comparison crosses papers; our own Coconut GPT-2 run in the same pipeline remains to be done.

The thoughts carry the information. With the thoughts removed, the same checkpoint scores 73.6%, about the level of the no-CoT baseline. Running six thoughts whose content is set to zero, which turns them into pause tokens [14], or replaced by noise gives 76.8% and 75.8%: the extra passes alone recover at most about 3 of the 25 points. What the thoughts contain, not the extra computation, carries the reasoning. The model was never trained with empty thoughts, so this test-time ablation cannot rule out that a model *trained* with pause tokens would learn to use the extra passes; that control remains open.

Writing the chain of thought hurts. Written CoT scores below no CoT on both splits (70.8% vs. 74.6% on test, 74.3% vs. 80.0% on validation). On test the two are about 1.4 standard errors apart, so the gap is suggestive rather than conclusive. Coconut found the two close on ProsQA (77.5% vs. 76.7% for GPT-2). Both observations fit the view that ProsQA's written reasoning forces the model to commit to one branch per sentence, so an early wrong step propagates. The logged test samples show one such failure: after following a wrong branch, the model repeats "Every zhorpus is a." until the length limit.

The gain appears once several steps are latent. Accuracy is flat while one or two steps are latent (74–76%) and then rises sharply: 86.3% at three thoughts, 93.3% at four, 97.3% at five. Section 7 discusses why.

Gold reasoning steps	3	4	5	6
Test questions	202	217	69	12
No CoT	74.3	74.7	75.4	75.0
Written CoT	73.8	68.7	68.1	75.0
Latent Mamba, fixed 6 thoughts	**99.0**	**99.1**	**98.6**	**100**
Latent Mamba, learned halting	98.5	**99.1**	97.1	**100**
thoughts used (mean)	2.99	3.13	3.35	3.42

Table 6: Test accuracy (%) by number of gold reasoning steps, and the number of thoughts learned halting uses for each group. The six-step group is small.

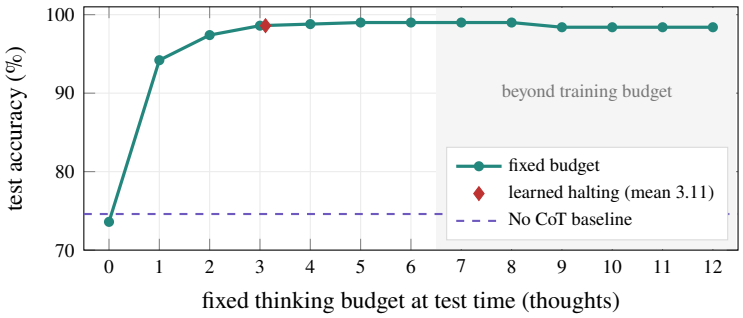


Figure 5: Test accuracy of the halting-stage checkpoint when the number of thoughts is fixed at test time. Training used at most 6 thoughts.

The gain holds on longer problems. Table 6 splits test accuracy by the number of gold reasoning steps. Written CoT is weakest on four- and five-step questions (68–69%); the latent model stays above 97% on every group.

One thought does most of the work, and the budget extrapolates. Figure 5 fixes the number of thoughts at test time. A single thought lifts accuracy from 73.6% to 94.2%, two reach 97.4%, and accuracy saturates at 99.0% from five thoughts on. Budgets above the six used in training lose at most 0.6 points up to twelve thoughts. During the curriculum, accuracy rose only once three or more steps were latent, yet at test time one thought already recovers most of the gain. The halting stage trains the answer after every number of thoughts from 1 to 6 at once, and it lifted validation accuracy from 97.0% to 99.3% within one epoch, so it may also teach the model to compress its search into fewer steps. We did not evaluate the pre-halting checkpoint at small budgets, so we cannot yet separate the two effects.

Learned halting finds the knee of the curve but adapts only weakly. The halting head stops after 3.11 thoughts on average and scores 98.6%, half the thoughts of the fixed mode for 0.4 points (two questions). A fixed budget of three scores the same 98.6%, so on this task the learned rule matches, but does not beat, a well-chosen fixed budget; its value is that it finds this budget without a sweep. Its thought count rises only slightly with difficulty, from 2.99 on three-step questions to 3.42 on six-step ones (Table 6). The convergence rule never fired at tolerance 10^{-2} and always used all six thoughts: consecutive thoughts keep changing by more than 1%, so the trajectory does not settle to a fixed point within the training budget.

Cost. The two baselines took about 44 and 41 minutes on one H100 NVL, including validation, and the full 55-epoch latent run took 14.0 hours. Each thought is one more sequential pass through all 24 layers, so throughput falls as the curriculum adds thoughts (Table 7): from 1.06 batches per second at stage 0 to 0.50 at six thoughts, roughly 0.18 seconds per thought per batch of 32. The thought steps process one token per example and are cheap in arithmetic; we attribute most of their cost to per-layer kernel launches. Compiling the thought step (Section 5) targets this cost; its effect on training throughput has not been measured yet.

6.3 Inference memory and latency

We measure Mamba-2 130M and GPT-2 124M on the same GPU in one session: batch 1, a 512-token prompt, bf16 weights, and n thoughts followed by one answer token (Table 8). Weights are random, which does not affect memory

Stage	0	1	2	3	4	5	6	final	halting
Thoughts	0	1	2	3	4	5	6	6	6 (scored 1–6)
Batches / s (batch 32)	1.06	0.92	0.82	0.71	0.59	0.57	0.50	0.53	0.55

Table 7: Training throughput by stage for Mamba-2 130M on one H100 NVL, with activation checkpointing.

	Thoughts n	0	16	64	256	480	1,024	4,096
State / KV cache (MiB)	Latent Mamba	18.2	18.2	18.2	18.2	18.2	18.2	18.2
	GPT-2	18.0	18.6	20.3	27.0	34.9	–	–
Peak extra memory (MiB)	Latent Mamba	80.0	97.4	97.4	97.4	97.4	97.4	97.4
	GPT-2	34.5	34.5	34.5	43.1	43.1	–	–
Latency (s)	Latent Mamba	0.035	0.066	0.161	0.544	0.988	2.07	8.10
	GPT-2	0.015	0.116	0.446	1.75	3.26	–	–

Table 8: Inference after a 512-token prompt on one GPU (batch 1, bf16). Latent Mamba uses fused kernels and CUDA graphs; GPT-2 is the Hugging Face implementation without CUDA graphs or compilation. "–": beyond GPT-2's 1,024-position context.

or time; latency is the median of repeated runs. Latent Mamba uses the fused scan, the fused single-token convolution and CUDA graphs (the single-step SSM update used the reference in this run). GPT-2 uses the Hugging Face implementation with its KV cache but *without* CUDA graphs or compilation, so it is the less optimised of the two.

The state is constant. Latent Mamba's state stays at 18.2 MiB from zero to 4,096 thoughts. This is twice the 9.25 MiB computed from Equation (5) because the implementation keeps the SSM state in float32 (Section 4.4); only the convolution window is bf16. GPT-2's cache grows by 36 KiB per token and passes the Mamba state after about seven thoughts. At 480 thoughts it is 1.9× larger, and GPT-2 cannot think beyond its 1,024-position context at all.

A thought costs 2.0 ms, a GPT-2 step 6.8 ms. Beyond the prompt, Latent Mamba's latency grows by 2.0 ms per thought, from 16 to 4,096 thoughts alike; GPT-2's grows by 6.8 ms per position, so at 480 thoughts it takes 3.26 s against 0.99 s. A thought updates a fixed-size state in one replayed CUDA graph, while each GPT-2 step attends over a growing cache. Part of the gap is optimisation, not architecture: a Transformer with CUDA graphs and a static cache would be faster than measured here, a comparison we have not run. GPT-2 reads the prompt faster (15 against 35 ms). The pure PyTorch implementation took about 21 ms per thought in an earlier session on an H100 NVL.

Peak memory favours GPT-2 at this scale. Peak memory above the weights is 80–97 MiB for Latent Mamba at every budget, against 35–43 MiB for GPT-2. The peak is set by the 512-token prompt pass, not by thinking. At six thoughts the KV cache a Transformer adds is negligible, so the constant-memory advantage matters for reasoning much longer than ProsQA's.

7 Discussion: why does latent reasoning help here?

The thought ablations of Table 5 show that the content of the thoughts matters. Why it helps on this task is less certain; the following explanations are hypotheses.

Search without committing. A written chain must choose one node of the graph per sentence. A continuous thought is a vector and can represent several candidate nodes at once, narrowing them over successive thoughts, which resembles breadth-first search. Coconut observed exactly this in GPT-2 on ProsQA: switched back to language after one or two thoughts, the model spreads probability over several candidate next nodes rather than committing to one [19]. The shape of our curve supports the same reading: little changes while only one or two steps are latent, and accuracy rises once three or more consecutive steps can stay in latent space, where keeping alternatives alive pays off.

Depth on demand. Each thought runs the full nonlinear stack once more, so six thoughts add 144 layers of computation on top of the prompt pass. Within a single pass, Mamba's state update is linear (Equation (1)); the thought loop supplies the nonlinear depth that path-finding needs.

A large fixed-size workspace. The 130M model carries a $24 \times 64 \times 128$ state in each of 24 layers. Thoughts read the graph facts held in this state and refine them in place, and nothing accumulates, which suits a task whose input is large (hundreds of tokens of facts) and whose reasoning is short.

Alternative explanations. Three confounds were open; the test results settle one and narrow another. (i) Training length: the latent model trained for 55 epochs and the baselines for 5. No CoT peaked at its first epoch and declined afterwards, which makes "more training" a weak explanation, but only the 55-epoch control settles it. (ii) Extra computation: zero and noise thoughts at test time recover at most 3 of the 25 points (Table 5), so extra passes are not sufficient for a model trained with informative thoughts. Whether a model trained with pause tokens could learn to use them is still open [14, 28]. (iii) Checkpoint selection: test accuracy confirms the validation result for Latent Mamba, and the baselines fall further on test, so selection does not explain the gap.

8 Related work

Latent reasoning. Coconut [19] introduced continuous thoughts and observed breadth-first-search-like behaviour. Implicit CoT [11] internalises reasoning steps through a curriculum. Pause tokens [14] and filler tokens [28] add compute without content; Pfau et al. show that such tokens can still carry hidden computation, which is why our zero-thought ablation is an informative control rather than a trivial one. Zhu et al. [39] survey the area and distinguish vertical from horizontal recurrence.

Recurrent depth. Universal Transformers [10] share one block across depth, optionally with per-position ACT halting, deep equilibrium models [3] solve for a layer's fixed point by root-finding, and recurrent networks trained on easy instances can extrapolate to harder ones by iterating longer [31]. Geiping et al. [13] train a depth-recurrent Transformer with randomly sampled iteration counts and obtain zero-shot per-token adaptive exit. RD-VLA [33] stops when the latent state converges. SST V2 [2] streams a latent state across positions in a Transformer and deliberates per position. Wang and Reid [36] place Mamba-2 hybrid blocks inside a tiny recursive reasoning model [20]. They leave open whether the recursion can be folded into the SSM's own state updates; Proposition 1 and Latent Mamba give one such folding.

Adaptive computation and SSM expressivity. We use ACT [15] and PonderNet [4] directly. Linear and diagonal SSMs such as S4 and Mamba, at fixed depth, cannot track state for problems such as permutation composition [25]; with only positive eigenvalues they cannot even solve parity, and allowing negative eigenvalues helps [16]. Thought steps offer a complementary route: more nonlinear passes rather than a richer transition.

Hybrid models. Hybrid architectures interleave attention with Mamba layers to restore exact recall [22, 35]. Latent Mamba is compatible with them: thoughts can update the SSM layers while being kept out of the attention layers' KV cache, so memory stays constant in the thinking budget.

Benchmarks. The pipeline also prepares ProntoQA [30] and GSM8k [8] in implicit CoT's augmented form [11].

9 Limitations

Narrow evidence: one synthetic dataset, one seed and one model size. Test graphs are new but share the training distribution, and ProsQA is close to saturated, so it cannot separate strong methods. Coconut GPT-2 trained in our pipeline, a pause-token model trained with empty thoughts, and a no-CoT model trained for 55 epochs are still missing. **Halting:** the learned head matches a fixed budget of three but adapts little to difficulty, and the convergence rule never fires; ProsQA's narrow range of difficulty (3 to 6 steps) may be part of the reason. **Benchmark scope:** latency was measured at batch 1, for one model size, against a Transformer without CUDA graphs or compilation, so the speed advantage at larger batches, larger models or against an optimised Transformer is untested. **Fixed-size memory cuts both ways:** thinking cannot restore prompt details the state has already lost, so recall-heavy tasks may need hybrid attention layers, with thoughts kept out of their KV cache. **Interpretability:** thoughts are not text, and decoding them through W is only a probe.

10 Conclusion

In a selective SSM, extra depth with a shared state is the same thing as extra time. Latent Mamba uses this to let a Mamba-2 model think in continuous space for as long as each input requires, at constant memory and with well under 1% more parameters. Thoughts reuse the backbone unchanged, so the method needs no new kernels, trains from public pretrained checkpoints, and scores every thinking budget in one batched pass during halting training, which a growing KV cache would make expensive.

The experiment supports the idea on held-out data. On ProsQA, a pretrained 130M Mamba-2 fine-tuned with the curriculum answers 99.0% of test questions while reasoning entirely in latent space, against 70.8% when the same model writes its reasoning and 74.6% when it answers directly, and in the range Coconut reports for GPT-2. The thoughts carry the reasoning: with their content zeroed or replaced by noise, accuracy falls to 76–77%, close to answering without thoughts. A single thought already recovers most of the gain, accuracy saturates at five, and twice the training budget costs under a point. The learned halting head stops after about three thoughts at almost no loss of accuracy, halving inference compute. On GPU, the state stayed at 18.2 MiB for budgets up to 4,096 thoughts, a length GPT-2 cannot reach, and with fused kernels and CUDA graphs a thought took 2.0 ms against 6.8 ms for a GPT-2 decoding step.

The results also mark the limits of the current evidence. Halting finds a good budget but matches a fixed budget of three rather than adapting strongly to each question. At this scale, peak memory is still higher than GPT-2's, the latency comparison is against a Transformer without CUDA graphs, and at ProsQA's reasoning length the constant-memory advantage is structural rather than yet practical. Everything rests on one seed of one synthetic task. The next steps are the matched Coconut GPT-2 baseline; trained pause-token and training-length controls; more seeds and model sizes; a latency comparison against an equally optimised Transformer; tasks with long reasoning, where a constant-size state should matter most; and ProntoQA and the harder GSM8k benchmark, where Coconut trailed written chain-of-thought and Latent Mamba faces its most demanding test.

References

[1] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, S. Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. EMNLP, 2023.

[2] T. Aviss. State Stream Transformer (SST) V2: Parallel training of nonlinear recurrence for latent space reasoning. arXiv:2605.00206, 2026.

[3] S. Bai, J. Z. Kolter, V. Koltun. Deep equilibrium models. NeurIPS, 2019.

[4] A. Banino, J. Balaguer, C. Blundell. PonderNet: Learning to ponder. 8th ICML Workshop on Automated Machine Learning, 2021. arXiv:2107.05407.

[5] S. Black, S. Biderman, E. Hallahan, et al. GPT-NeoX-20B: An open-source autoregressive language model. Proceedings of BigScience Episode #5, Workshop on Challenges & Perspectives in Creating Large Language Models, 2022.

[6] G. E. Blelloch. Prefix sums and their applications. Technical report CMU-CS-90-190, Carnegie Mellon University, 1990.

[7] T. Chen, B. Xu, C. Zhang, C. Guestrin. Training deep nets with sublinear memory cost. arXiv:1604.06174, 2016.

[8] K. Cobbe, V. Kosaraju, M. Bavarian, et al. Training verifiers to solve math word problems. arXiv:2110.14168, 2021.

[9] T. Dao, A. Gu. Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality. ICML, 2024.

[10] M. Dehghani, S. Gouws, O. Vinyals, J. Uszkoreit, Ł. Kaiser. Universal transformers. ICLR, 2019.

[11] Y. Deng, Y. Choi, S. Shieber. From explicit CoT to implicit CoT: Learning to internalize CoT step by step. arXiv:2405.14838, 2024.

[12] L. Gao, S. Biderman, S. Black, et al. The Pile: An 800GB dataset of diverse text for language modeling. arXiv:2101.00027, 2020.

[13] J. Geiping, S. McLeish, N. Jain, et al. Scaling up test-time compute with latent reasoning: A recurrent depth approach. NeurIPS, 2025.

[14] S. Goyal, Z. Ji, A. S. Rawat, A. K. Menon, S. Kumar, V. Nagarajan. Think before you speak: Training language models with pause tokens. ICLR, 2024.

[15] A. Graves. Adaptive computation time for recurrent neural networks. arXiv:1603.08983, 2016.

[16] R. Grazzi, J. Siems, A. Zela, J. K. H. Franke, F. Hutter, M. Pontil. Unlocking state-tracking in linear RNNs through negative eigenvalues. ICLR, 2025.

[17] A. Gu, K. Goel, C. Ré. Efficiently modeling long sequences with structured state spaces. ICLR, 2022.

[18] A. Gu, T. Dao. Mamba: Linear-time sequence modeling with selective state spaces. COLM, 2024. arXiv:2312.00752.

[19] S. Hao, S. Sukhbaatar, D. Su, et al. Training large language models to reason in a continuous latent space. COLM, 2025. arXiv:2412.06769.

[20] A. Jolicoeur-Martineau. Less is more: Recursive reasoning with tiny networks. arXiv:2510.04871, 2025.

[21] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, Y. Iwasawa. Large language models are zero-shot reasoners. NeurIPS, 2022.

[22] O. Lieber, B. Lenz, H. Bata, et al. Jamba: A hybrid Transformer-Mamba language model. arXiv:2403.19887, 2024.

[23] I. Loshchilov, F. Hutter. Decoupled weight decay regularization. ICLR, 2019.

[24] E. Martin, C. Cundy. Parallelizing linear recurrent neural nets over sequence length. ICLR, 2018.

[25] W. Merrill, J. Petty, A. Sabharwal. The illusion of state in state-space models. ICML, 2024.

[26] P. Micikevicius, S. Narang, J. Alben, et al. Mixed precision training. ICLR, 2018.

[27] M. Nye, A. J. Andreassen, G. Gur-Ari, et al. Show your work: Scratchpads for intermediate computation with language models. arXiv:2112.00114, 2021.

[28] J. Pfau, W. Merrill, S. R. Bowman. Let's think dot by dot: Hidden computation in transformer language models. COLM, 2024. arXiv:2404.15758.

[29] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever. Language models are unsupervised multitask learners. OpenAI technical report, 2019.

[30] A. Saparov, H. He. Language models are greedy reasoners: A systematic formal analysis of chain-of-thought. ICLR, 2023.

[31] A. Schwarzschild, E. Borgnia, A. Gupta, F. Huang, U. Vishkin, M. Goldblum, T. Goldstein. Can you learn an algorithm? Generalizing from easy to hard problems with recurrent networks. NeurIPS, 2021.

[32] N. Shazeer. Fast transformer decoding: One write-head is all you need. arXiv:1911.02150, 2019.

[33] Y. Tur, J. Naghiyev, H. Fang, et al. Recurrent-Depth VLA: Implicit test-time compute scaling of vision-language-action models via latent iterative reasoning. arXiv:2602.07845, 2026.

[34] A. Vaswani, N. Shazeer, N. Parmar, et al. Attention is all you need. NeurIPS, 2017.

[35] R. Waleffe, W. Byeon, D. Riach, et al. An empirical study of Mamba-based language models. arXiv:2406.07887, 2024.

[36] W. Wang, F. Reid. Tiny recursive reasoning with Mamba-2 attention hybrid. ICLR 2026 Workshop on Latent & Implicit Thinking. arXiv:2602.12078, 2026.

[37] J. Wei, X. Wang, D. Schuurmans, et al. Chain-of-thought prompting elicits reasoning in large language models. NeurIPS, 2022.

[38] B. Zhang, R. Sennrich. Root mean square layer normalization. NeurIPS, 2019.

[39] R.-J. Zhu, T. Peng, T. Cheng, et al. A survey on latent reasoning. arXiv:2507.06203, 2025.